

Verifying Left-Leaning **Red-Black** Tree with Dafny

Sara Babaee¹, Qikang Liu², and Mohammad Mahdi Khodabandeh³

^{1,2,3}Simon Fraser University

December 6, 2023

1 Introduction: The Program

A Red-Black (RB) tree is a “dynamic set” data structure, implemented as a balanced binary search tree allowing for efficient operations; most notably: search, insertion, and deletion. Each node, in addition to the *key* and *incidence pointers*, keeps an additional bit that determines its *color*: red or black. Note that we can equivalently view the links (edges) in the tree as colored rather than the nodes; one just assigns the color of a node to the link attaching it to its parent. This data structure imposes some invariant (using the colors) on the structure of the tree such that the height of this tree maintains $O(\log(n))$ length, where n is the number of elements in this set. The small height of the tree makes the operations performed on the tree extremely efficient.

The standard implementation of this data structure, found in textbooks (e.g. CLRS [1]) has been simplified by the notion of *Left-Leaning* RB Tree (LLRB) [4]. The implementation is found in [2]. Due to the simplified implementation, the task of verifying [3] LLRBs is easier. In this project, we verify the correctness of an implementation of LLRB inspired by what has been done in [3], and we further extend it by introducing methods such as “search”. Our code adopts Dafny’s functional programming feature, emphasizing immutable variables. The Dafny version used for verification is **Dafny 3.x**.

2 Properties Verified

In this section, we give a high level description of the most important properties that we verified and merely present a big picture. The detailed verification can be found in the implementation.

We implement, along with other functions, the following major functions: search, insert, and delete. An LLRB guarantees the following invariants after the execution of each of these functions ([4]):

- No path from the root to an arbitrary leaf contains two consecutive red links.
- Each path from the root to an arbitrary leaf has the same number of black links.

The second invariant is in fact stronger. It holds not only for the root but also for each node in the tree. These invariants impose structural restrictions on the tree. For example, each node must follow a *good*

coloring: (i) if a node is red, both its children must be black (ii) if the right child of a black node is red so is its left child (left-leaning).

Given the invariants, we have a predicate that tells us what it exactly means to be an LLRB tree. The property of being LLRB is (recursively) defined as follows:

$$\text{isLLRB}(T) \iff T = \emptyset \vee (\text{isLLRB}(T.\text{left}) \wedge \text{isLLRB}(T.\text{right}) \wedge \text{balanced}(T) \wedge \text{BST}(T) \wedge \text{goodColor}(T))$$

Each of the predicates above are defined as follows:

- $\text{balanced}(T)$: The black height of the left and right sub-tree is the same. Here black height means the number of black links from root to some leaf. Recall that this number is the same for each leaf.
- $\text{BST}(T)$: T is a binary search tree. That is, for each node, any key in the left sub-tree is less than or equal to the key in the node, and the key in the node is less than or equal to any key in the right sub-tree.
- $\text{goodColor}(T)$: No two consecutive red links. Moreover, T is left-leaning. That is $\text{color}(T.\text{right}) = \text{Red} \implies \text{color}(T.\text{left}) = \text{Red}$

So our pre and postconditions after our major tree operations (e.g., insert, delete) are the same: isLLRB . More precisely, let T be an LLRB, and let $OP \in \{\text{search}, \text{insert}, \text{delete}\}$ be an operation on the tree. This operation takes a tree T and a key x as input. The following must hold:

$$\underbrace{\text{isLLRB}(T)}_{\text{requires}} \implies \underbrace{\text{isLLRB}(OP(T, x))}_{\text{ensures}}.$$

Trivially, an identity operation has this property. So, in addition to this check, we need to verify that the operation modified the tree in the desired way. For instance, in case of insert, we also check that the element we wanted to was inserted into the tree. That is, $\text{set}(\text{insert}(T, x)) = \text{set}(T) \cup \{x\}$, where x is the key that was added in insert. Similarly, in case of delete, it holds that $\text{set}(\text{delete}(T, x)) = \text{set}(T) - \{x\}$. Needless to say that the implementation is more involved than this as we need to implement auxiliary and helper functions, such as rotation, which themselves need some guarantees with their verification.

3 Proof

3.1 Search

In an LLRB, the search function takes a value x and an LLRB tree, t , as input to locate a node with the key x within t . It commences by comparing the root's key with x . If x is greater, the function recursively explores the right subtree; if smaller, it navigates through the left subtree. When x matches the root's key, the function returns the root as the node containing x . Furthermore, if the search encounters an empty leaf, it returns an Empty LLRB, indicating that the value x does not exist within t .

3.1.1 Specifications

The sole **precondition** required by the search function is that the input tree complies with the LLRB structure. Regarding the **postconditions**, it guarantees: (1) it returns an LLRB, (2) if the value x exists in the set of keys within the input tree, it returns a node holding that value, and (3) if the function returns `Empty`, it signifies that the value x does not exist within the set of keys belonging to the input tree.

3.2 Insertion

The `insert` function within an LLRB structure aims to add a new node while preserving all LLRB properties. It takes a value, x , and an LLRB tree, t , ensuring that after insertion, the resulting tree remains a valid LLRB containing the new element.

Similar to a conventional BST, the `insert` function recursively searches for the value x in t and eventually, either locates it or determines its suitable insertion point. In the second case, it adds x in a way that preserves the balanced property and then fixes the `goodColor` property by rotations.

3.2.1 Specifications

According to the definition of the `insert` function, it requires a single **precondition**: the provided t must qualify as an LLRB. It also needs to ensure two **postconditions**: (1) the resulting tree should retain its LLRB properties, and (2) it should include the same set of data (keys) as the input tree (t), along with the added value x .

3.2.2 Functionality

If the value x does not exist in the t , the `insert` function initially adds x as a red node to prevent a violation of the balanced property. However, this insertion might potentially disrupt the `goodColor` property, a situation addressed within the `checkColors` function, working from the bottom to the top of the tree.

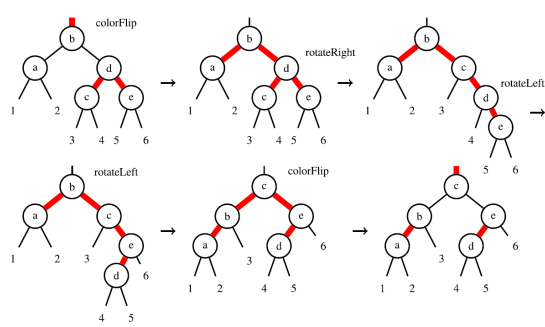
The `checkColors` function fixes the `goodColor` properties for the input node t by (1) ensuring that t lacks a red right child and a black left child through the `leftLean` function, and (2) confirming the absence of consecutive red edges to the left of t using the `avoidConsecReds` function. Notably, cases involving a red right child need not undergo inspection due to the preemptive application of the `colorFlip` function. This function, executed from root to inserted leaf initially, alters the color of both children to black if they were red, preserving the tree's black height while changing the node's color to red.

In this approach, considering the possibility of both the root and its left child being red after insertion, all these processes are consolidated within an `internalInsert` function. This function needs two crucial **postconditions**: (1) if the root color is black, the resulting tree qualifies as an LLRB, and (2) if the root color is red, the resulting tree aligns with a `weakLLRB`, signifying an LLRB without the constraint of disallowing two consecutive red edges from the root. Subsequently, in the user-accessible `insert` function, we employ the `blacken` function to alter the color of the root to black. This transformation ensures that the tree consistently adheres to the LLRB properties. [3]

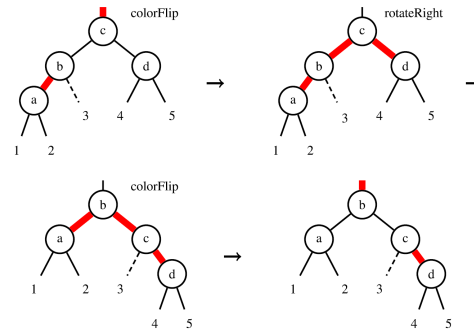
3.3 Deletion

If the node to delete is a leaf node, we need to delete it. If it is not a leaf node, we replace its value with the minimal value in the right subtree and delete that leaf node with the minimal value. Therefore, we only need to deal with the case of deleting a leaf node.

To ensure that all the paths from the root to an empty tree have the same number of black edges, we can only apply deletion to is NOT a 2-node (i.e. node without a red link from and to it). There are two cases while deleting value x in a LLRB t : $x < t.key$ and $x \geq t.key$. In both cases, when the root of the subtree we want to apply delete to is a 2-node, we need two corresponding methods `ensureRedLeft` and `ensureRedRight` for the two cases to ensure we do not apply delete to a 2-node.



(a) The worst case of function `ensureRedLeft` [3]



(b) The worst case of function `ensureRedRight` [3]

3.3.1 `ensureRedLeft`

While the target value to delete is in the left subtree, and the root of the left subtree is a 2-node, we need to do some transformations to make it a red node. This is what `ensureRedLeft` does. First, we apply `colorFlip` to the parent whose color is red because the delete function uses it by passing the current root which is red to it. Then we check whether two consecutive red links have been created. This happens when the left child of the right child is red. If so we do `rotateRight`. We also need to check whether the sibling link is red before rotation. If so, we do an additional `rotateLeft`. Finally, we do a `rotateLeft` to the root and a second `colorFlip` to ensure the invariant.

The **precondition** requires the input tree to be an LLRB, the root color to be red, and the left child to be a 2-node. There are two **postconditions** corresponding to the case when a single color flip suffices and the case when the whole sequence of rotations and a second color flip are needed. The **postcondition** also ensures the output tree is also an LLRB with the set of keys and black heights unchanged. The code `card(res.left) < card(t)` is used to ensure the termination of `delete_`. Because the `delete_` will then be applied to `res.left` recursively, we need to ensure that the size of the tree decreases in the next call of `delete_`.

3.3.2 `ensureRedRight`

The cases in which the element to be deleted is greater than, or equal to the root key must be dealt with together. This is because deleting the root is equal to deleting the minimal element in the right subtree. First, we check whether there is an isolated red link to the left. If so, we rotate the tree to the right. This way ensures that the root of the right child is red and avoids deleting an internal node with an empty right child and a non-empty left one. If no rotation is done, we check whether the right child is a 2-node.

If so, we do `ensureRedRight` to make it red. The `ensureRedRight` firstly applies a `colorFlip` to the parent. Because the left child is not red, the right child must not be red because it is left-leaning, and the parent is not a 2-node, the parent must be red. After `colorFlip`, we check whether two consecutive red links have been created. If so, a `rotateRight`, followed by a second `colorFlip`, solves the problem.

After this, we will apply `delete_` to the right subtree. The root of the right subtree may be a 3-node whose left child is black and the right child is red, which is not allowed in LLRB. We call this temporary RB Tree `mirrorLLRB`. And the `delete_` can accept both LLRB and `mirrorLLRB`.

The **precondition** requires the input tree to be an LLRB, the root color to be red, and the right child to be a 2-node. There are two **postconditions** corresponding to the case when a single color flip suffices and the case when a rotation and a second color flip are needed. The **postcondition** also ensures that the output tree's set of keys and black heights are preserved. And `card(res.left) < card(t)` is still used to ensure the termination of `delete_`. One difference to `ensureRedLeft` is that the output tree might not be an LLRB and can contain `mirrorLLRB`s.

3.3.3 The delete function

With `ensureRedLeft` and `ensureRedRight`, we can now apply deletion to a non-2-node. Based on the relationship between the key to delete and the key of the root, there are two cases: $x < t.key$ and $x \geq t.key$.

In the case $x < t.key$, we first check whether the root of the left subtree is a 2-node and do `ensureRedLeft` if it is. Then we apply deletion to the left subtree recursively.

In the case $x \geq t.key$, we first check whether there is an isolated red link to the left. If so, we rotate the tree to the right. Otherwise, we apply `ensureRedRight` if the root of the right subtree is a 2-node. After this, we get a new root `t1`. Then if $x > t1.key$, we apply deletion to the right subtree recursively. If $x == t1.key$, we check whether `t1` is a leaf node. If so, we return `Empty` which is the result of deletion. Otherwise, we replace `t1`'s key with the minimal value in the right subtree and apply deletion to the right subtree recursively to delete this minimal value.

After applying `delete_` to the left or right subtree, there will be an isolated right link (right-leaning `mirrorLLRB`) if the result of deleting in the left subtree is black-rooted, and the right subtree is still red-rooted because no modification. Here we use `leftLean` to fix it by doing `rotateLeft` if there is an isolated right link.

The **precondition** for `delete_` is that `t` can be an LLRB or a `mirrorLLRB` (right-leaning). And `t` is not a 2-node, there must be a red link linked to it. The **postcondition** is that the returned tree must be an LLRB, the black height is preserved, the target value is removed from the key set, and the number of nodes decreases in the next recursive call to prove termination.

In addition, the initial tree's root may be a 2-node and we cannot apply `delete_` on it. To solve this problem, we create a wrapper function `delete`, which checks the tree and paints the root to red if it is a 2-node. Then it applies `delete_` to the root and paints it back to black after `delete_` returns. Also, we introduce intermediate assertions, calculational proofs, and lemmas to help Dafny prove complex verification conditions.

4 Conclusion

We formally verified a more limited and simpler variant of the Red-Black Tree, namely the Left-Leaning Red-Black Tree (LLRB). Compared to the traditional Red-Black Tree, LLRB features a reduced number of insertion and deletion cases, simplifying its structure. Our primary focus involved verifying the functionality of search, insertion, and deletion within LLRB. Achieving this required the creation of specific functions like `rotateLeft`, `leftLean`, and `ensureRedLeft` to implement and preserve LLRB's properties. We also create many predicates, lemmas, and ghost functions to help us prove their correctness through preconditions, postconditions, and assertions.

However, it is essential to note a potential limitation. The verification process for deletion proved significantly more time-consuming than other methods due to its complicated nature. In some cases, particularly with specific implementations and computers, Dafny returned a time-out error during the verification of deletion due to its complexity. Additionally, we did not provide proof for the time or space complexities concerning the mentioned functions.

5 Division of Work

Teammate	Code	Report
Sara	Search, Insert	Proof, Conclusion
Qikang	Delete	Proof, Conclusion
Mohammad	Lemmas	Program, Properties

References

- [1] Thomas H Cormen, Charles E Leiserson, Ronald L Rivest, and Clifford Stein. *Introduction to algorithms*. MIT press, 2022.
- [2] Petar Maymounkov, Vadim Vygonets, Ian Smith, and Martin Bruse. Gollrb. <https://github.com/petar/GoLLRB>, 2019.
- [3] Ricardo Peña. An assertional proof of red-black trees using dafny. *Journal of Automated Reasoning*, 64(4):767–791, 2020.
- [4] Robert Sedgewick. Left-leaning red-black trees. In *Dagstuhl Workshop on Data Structures*, volume 17, 2008.